

**CEBU INSTITUTE OF TECHNOLOGY
UNIVERSITY**

COLLEGE OF COMPUTER STUDIES

Software Design Description

for

CapVault: A Capstone Project Tracking and Archival Management System

Change History

Version	Date	Author/s	Description
1.0	5/22/26	Sia, Fernandez, Tabanas, Taghoy	Initial SRS draft for CapVault

Preface

This Software Design Description document presents the technical design of CapVault: A Capstone Project Tracking and Archival Management System. It describes the system architecture, user interface design, frontend components, backend components, object-oriented components, sequence diagrams, and data design used to support the system requirements defined in the Software Requirements Specification.

This document is intended for the project team, adviser, and other technical reviewers who need to understand how the system will be structured and implemented.

Table of Contents

1. INTRODUCTION.....	4
1.1. PURPOSE.....	4
1.2. SCOPE.....	4
1.3. DEFINITIONS AND ACRONYMS.....	5
1.4. REFERENCES.....	6
2. ARCHITECTURAL DESIGN.....	7
3. DETAILED DESIGN.....	8
Module 1: Capstone Tracking and Class Record Management.....	8
1.1 Import Class Record.....	8
User Interface Design:.....	8
Front-end component(s):.....	9
Back-end component(s):.....	9
Object-Oriented Components:.....	10
Data Design:.....	12
1.2 Manage Capstone Groups and Deliverables.....	13
User Interface Design:.....	13
Front-end component(s):.....	14
Back-end component(s):.....	14
Object-Oriented Components:.....	15

Data Design:.....	17
Module 2: Submission and Version Control.....	18
2.1 Submit Capstone Deliverable.....	18
User Interface Design:.....	18
Front-end component(s):.....	19
Back-end component(s):.....	19
Object-Oriented Components:.....	20
Data Design:.....	22
2.1 Review Submission and Add Remarks.....	23
User Interface Design:.....	23
Front-end component(s):.....	24
Back-end component(s):.....	24
Object-Oriented Components:.....	25
Data Design:.....	27
Module 3: Archival Vault and File Integrity Management.....	28
3.1 Archive Submitted Document.....	28
User Interface Design:.....	28
Front-end component(s):.....	29
Back-end component(s):.....	29
Object-Oriented Components:.....	30
Data Design:.....	32
3.2 Generate and Verify SHA-256 Hash.....	33
User Interface Design:.....	33
Front-end component(s):.....	34
Back-end component(s):.....	34
Object-Oriented Components:.....	35
Data Design:.....	37
Module 4: Search, Retrieval, and Reporting.....	38
4.1 Search and Retrieve Archived Records.....	38
User Interface Design:.....	38
Front-end component(s):.....	39
Back-end component(s):.....	39
Object-Oriented Components:.....	40
Data Design:.....	42
4.2 Generate Dashboard and Reports.....	43
User Interface Design:.....	43
Front-end component(s):.....	44
Back-end component(s):.....	44
Object-Oriented Components:.....	45
Data Design:.....	47

1. Introduction

1.1. Purpose

This Software Design Description document describes the system design and technical architecture of CapVault: A Capstone Project Tracking and Archival Management System. It explains how the system components interact, how the functional requirements from the SRS are implemented, and how data flows through the system.

The document provides detailed descriptions of the frontend components, backend components, object-oriented components, sequence diagrams, and data design for each major transaction. Its purpose is to guide the development team in implementing a maintainable web-based system for capstone project tracking, document submission, version control, archival storage, file integrity verification, search, retrieval, and reporting.

1.2. Scope

CapVault is a web-based system designed to centralize capstone project monitoring and document archiving. The system allows Admin, Adviser, and Student users to manage class records, capstone groups, deliverables, submissions, adviser remarks, document versions, archived files, and reports.

The system supports the following core design areas:

- Google account validation and role-based access control
- Google Sheets class record import and configurable column mapping
- Capstone group and deliverable management
- Student submission through file upload or Google Drive link
- Submission version control and adviser review
- Archival storage with metadata
- PDF conversion for supported document formats
- SHA-256 file integrity verification
- Archive search, retrieval, dashboard summaries, reports, and activity logs

The system does not include grading computation, plagiarism detection, AI document evaluation, automated scoring, or full learning management system features.

1.3. Definitions and Acronyms

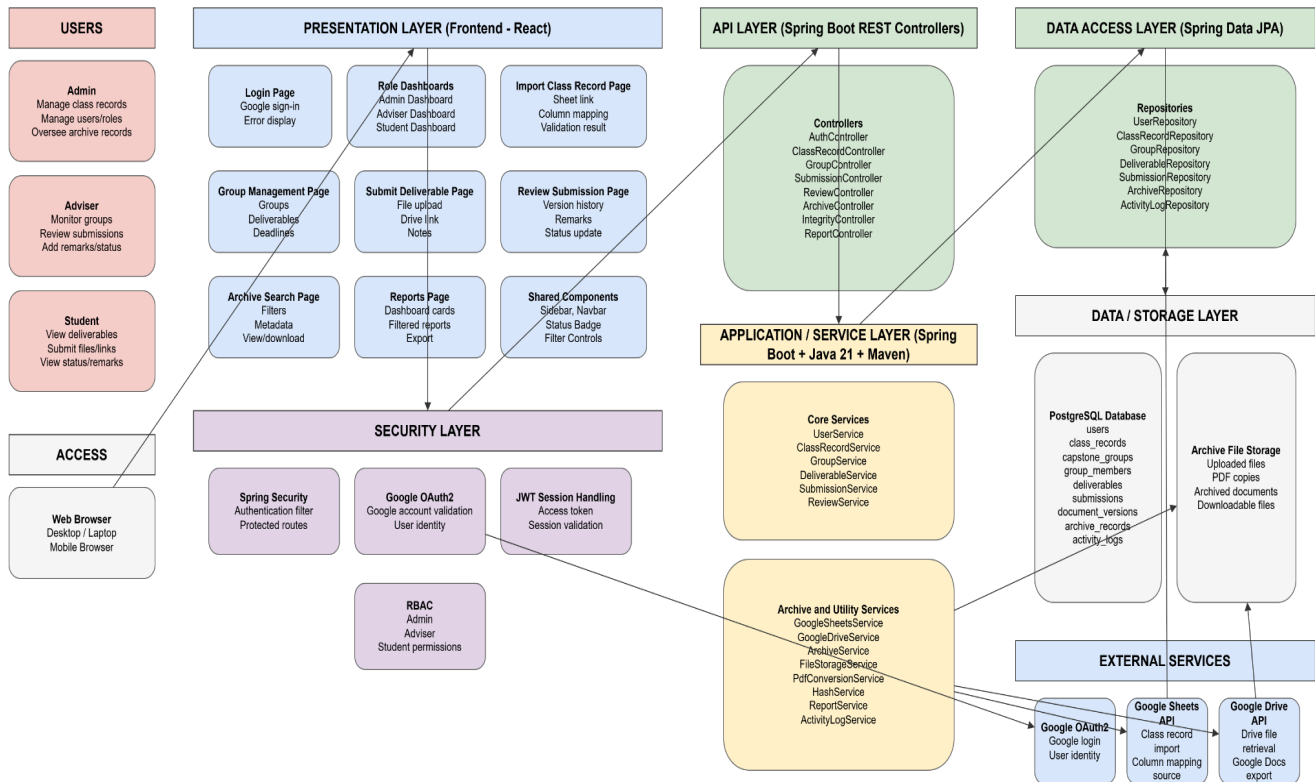
Term	Definition
Admin	The user role responsible for managing users, class records, system settings, and archive records.
Adviser	The user role responsible for monitoring assigned capstone groups, reviewing submissions, adding remarks, and updating submission status.
Student	The user role responsible for submitting capstone deliverables and viewing their group's submission history.
Capstone Project	A major academic project completed by students as part of their course requirement.
Deliverable	A required capstone output such as proposal, SRS, SDD, SPMP, final manuscript, presentation file, or source code record.
Archive	A controlled storage area where submitted capstone documents are preserved.
Metadata	Descriptive information attached to an archived document, such as project title, group code, adviser, deliverable type, version number, date submitted, archive date, and status.
Version Control	The process of saving updated submissions as separate versions instead of overwriting previous files.
RBAC	Role-Based Access Control.
OAuth2	Open Authorization 2.0, used for secure Google account login.
JWT	JSON Web Token, used for authenticated user sessions.
API	Application Programming Interface.
UI	User Interface.
DTO	Data Transfer Object.
CRUD	Create, Read, Update, Delete.
ERD	Entity Relationship Diagram.
SHA-256	Secure Hash Algorithm 256-bit, used for file integrity verification.
PDF	Portable Document Format.
SRS	Software Requirements Specification.
SDD	Software Design Description.

1.4. References

- CapVault Software Requirements Specification, Team 2526-sem2-it332-41.
- CapVault Project Proposal, Weeks 7–8, Team 2526-sem2-it332-41.
- CapVault Review of Related Literature, Existing Solutions Analysis, Gap Identification, and Traceability Matrix, Weeks 5–6, Team 2526-sem2-it332-41.
- Google Developers. (n.d.). Using OAuth 2.0 to access Google APIs.
<https://developers.google.com/identity/protocols/oauth2>
- Google Developers. (n.d.). Google Sheets API overview.
<https://developers.google.com/workspace/sheets/api/guides/concepts>
- Google Developers. (n.d.). Google Drive API overview.
<https://developers.google.com/workspace/drive/api/guides/about-sdk>
- React. (n.d.). React documentation. <https://react.dev/>
- Spring. (n.d.). Spring Boot documentation. <https://docs.spring.io/spring-boot/index.html>
- PostgreSQL Global Development Group. (n.d.). PostgreSQL documentation.
<https://www.postgresql.org/docs/>

2. Architectural Design

Architectural Design - CapVault



The CapVault system follows a client-server architecture. The presentation layer uses React and React Router to provide role-based pages for Admin, Adviser, and Student users. The frontend communicates with the backend through REST API requests.

The backend uses Spring Boot with Java 21 to handle business logic, authentication, authorization, file processing, Google API integration, archive management, and database communication. Spring Security, Google OAuth2, and JWT are used for authentication and role-based access control.

The file processing layer manages uploaded files, Google Drive document retrieval, PDF conversion, archived file storage, and SHA-256 hash generation. PostgreSQL stores user records, class records, capstone groups, deliverables, submissions, document versions, archive metadata, and activity logs.

External Google services are used for Google account validation, Google Sheets class record import, and Google Drive document retrieval.

3. Detailed Design

Module 1: Capstone Tracking and Class Record Management

1.1 Import Class Record

User Interface Design:

Wireframe - 1.1 Import Class Record

CapVault Import Class Record

Google Sheets Link / Spreadsheet ID Preview

Column Mapping Table

Sheet Column	→	System Field
Name	→	Student Name
Email	→	Student Email
Section	→	Section
Group	→	Group Code
Adviser	→	Adviser

Validation Results

Valid records: 42
Missing email: 1
Duplicate group code: 0

Import Cancel

Sidebar

- Dashboard
- Class Records
- Groups
- Submissions
- Archive
- Reports

The Import Class Record interface allows the Admin to connect a Google Sheets class record to CapVault. The page includes a Google Sheets link input, a preview button, a column mapping section, validation results, and action buttons for importing or canceling the import process.

Front-end component(s):

- **ClassRecordImportPage**
 - Description and purpose: Provides the interface for entering a Google Sheets link, previewing sheet columns, mapping columns to system fields, viewing validation results, and confirming class record import.
 - Component type or format: React Page Component.
- **ColumnMappingPanel**
 - Description and purpose: Displays sheet columns and allows the Admin to map them to CapVault fields such as group code, project title, software name, description, remarks, adviser/status, category, document links, student name, email, and section.
 - Component type or format: React Form Component.
- **ValidationResultPanel**
 - Description and purpose: Displays valid records, missing fields, duplicate entries, and import warnings before the Admin confirms the import.
 - Component type or format: React Display Component.

Back-end component(s):

- **ClassRecordController**
 - Description and purpose: Handles API requests for previewing Google Sheets data and importing mapped class records into the system.
 - Component type or format: REST Controller.
- **GoogleSheetsService**
 - Description and purpose: Connects to the Google Sheets API, retrieves sheet columns and rows, and returns sheet data for mapping and import.
 - Component type or format: Service Class.
- **ColumnMappingService**
 - Description and purpose: Validates required fields, maps Google Sheet columns to CapVault fields, and prepares valid class records for saving.
 - Component type or format: Service Class.
- **ClassRecordRepository**
 - Description and purpose: Saves and retrieves imported class record data from the PostgreSQL database.
 - Component type or format: Repository Interface.
- **ClassRecord**
 - Description and purpose: Represents imported class record data such as group code, project title, software name, description, remarks, adviser/status, category, and document links.
 - Component type or format: Entity Class.

Object-Oriented Components:

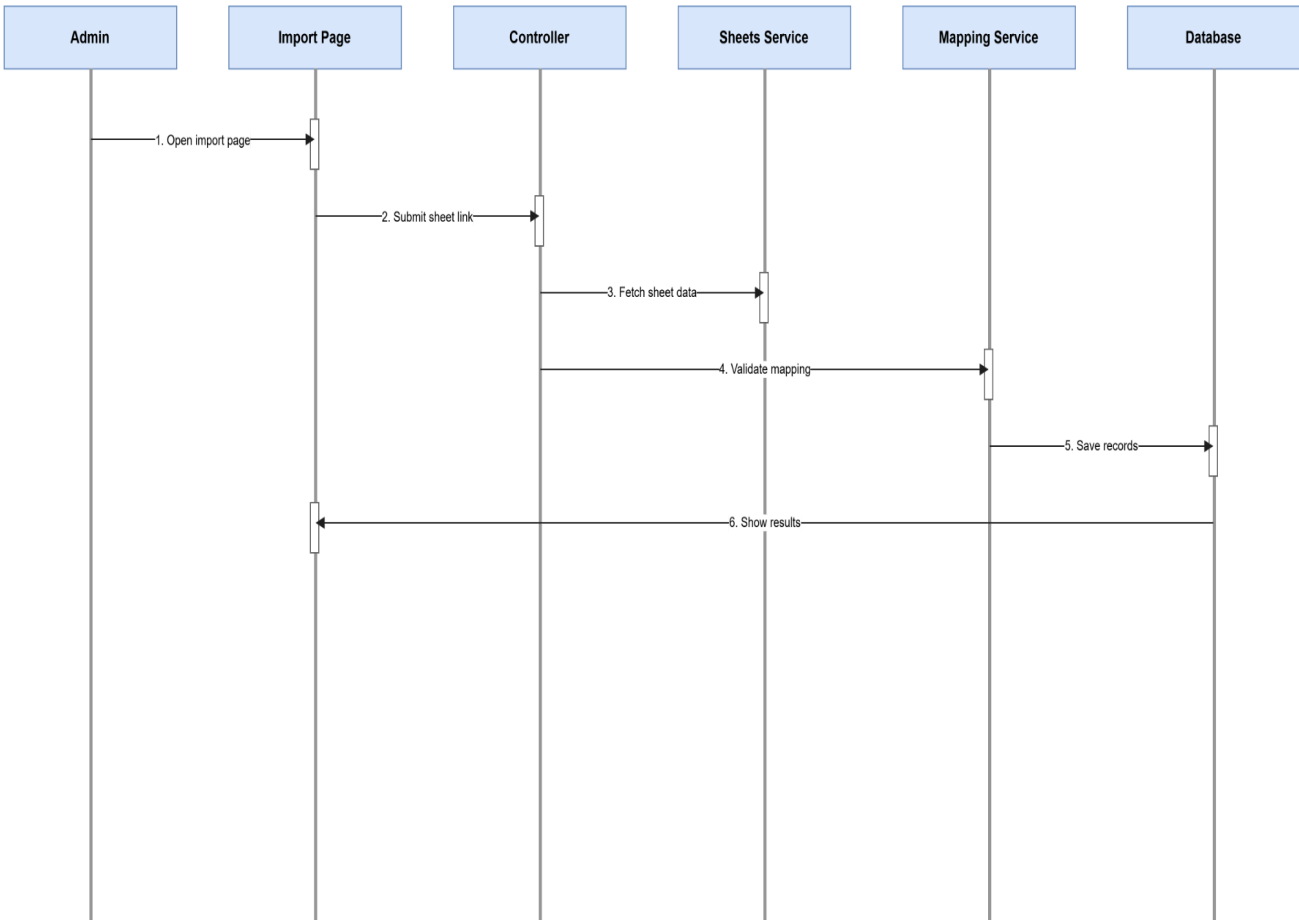
- Class Diagram:

Class - 1.1 Import Class Record



● Sequence Diagram:

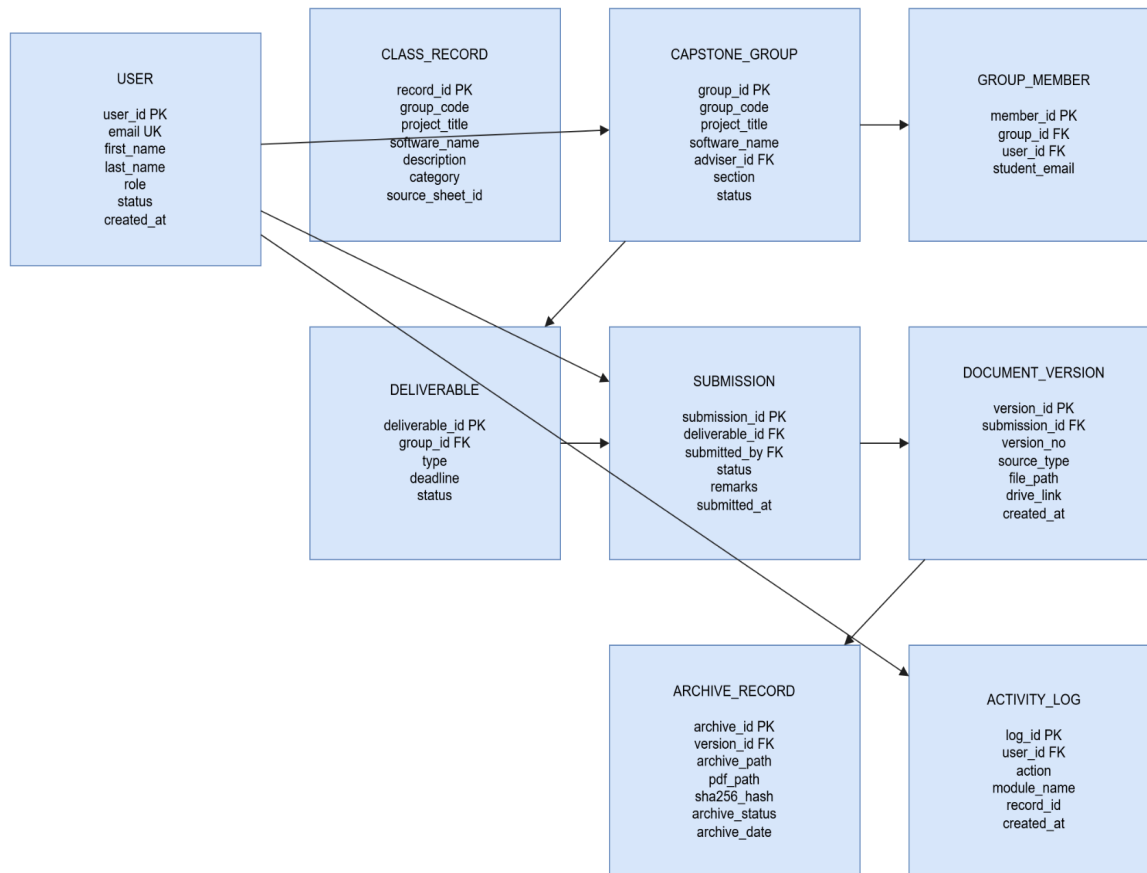
Sequence - 1.1 Import Class Record



Data Design:

- ERD or schema:

ERD - Global CapVault Schema



Relevant tables/entities: USER, CLASS_RECORD, CAPSTONE_GROUP, GROUP_MEMBER, and ACTIVITY_LOG.

1.2 Manage Capstone Groups and Deliverables

User Interface Design:

Wireframe - 1.2 Manage Groups and Deliverables

CapVault

Manage Capstone Groups and Deliverables

Search group / project / adviser

Group List

G01 | CapVault | Adviser A | 4 members
G02 | Project B | Adviser B | 3 members
G03 | Project C | Adviser A | 4 members

Group Details

Project Title: CapVault
Members: Student A, Student B, Student C
Adviser: Adviser A
Section: IT332

Deliverables

Proposal | Due: May 20 | Submitted
SRS | Due: May 23 | Missing
SDD | Due: May 24 | Under Review

Save Changes

Sidebar

Dashboard
Class Records
Groups
Submissions
Archive
Reports

The Manage Capstone Groups and Deliverables interface allows Admin and Adviser users to view capstone groups, project titles, members, assigned advisers, deliverables, deadlines, remarks, and current submission statuses.

Front-end component(s):

- **GroupManagementPage**
 - Description and purpose: Displays the group list, search/filter controls, group details panel, and deliverables table.
 - Component type or format: React Page Component.
- **GroupDetailsPanel**
 - Description and purpose: Shows selected group information such as group code, members, adviser, project title, software name, description, and category.
 - Component type or format: React Display Component.
- **DeliverablesTable**
 - Description and purpose: Displays assigned deliverables, deadlines, current statuses, and remarks for each group.
 - Component type or format: React Table Component.

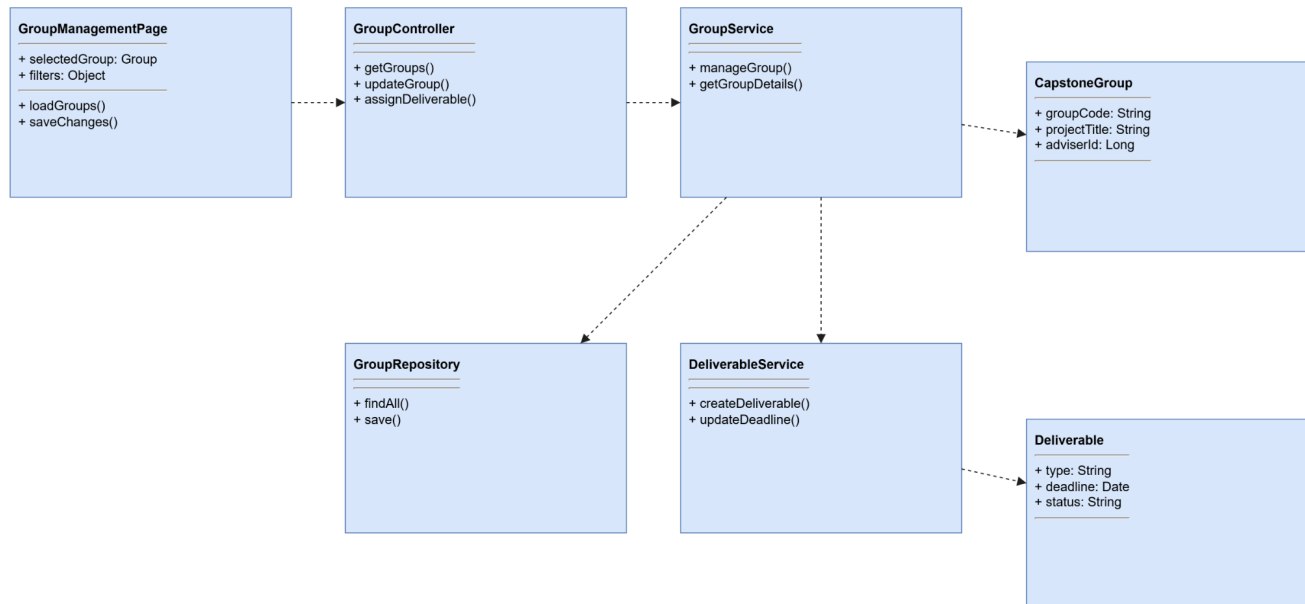
Back-end component(s):

- **GroupController**
 - Description and purpose: Handles API requests for retrieving, creating, updating, and managing capstone groups.
 - Component type or format: REST Controller.
- **GroupService**
 - Description and purpose: Processes group management logic, permission checks, group detail retrieval, and group updates.
 - Component type or format: Service Class.
- **DeliverableService**
 - Description and purpose: Manages deliverable records, deadlines, status updates, and assigned deliverables per group.
 - Component type or format: Service Class.
- **GroupRepository**
 - Description and purpose: Retrieves and stores capstone group records in the database.
 - Component type or format: Repository Interface.
- **CapstoneGroup**
 - Description and purpose: Represents a capstone group with group code, project title, software name, adviser, section, and status.
 - Component type or format: Entity Class.
- **Deliverable**
 - Description and purpose: Represents required outputs assigned to capstone groups, including type, deadline, and current status.
 - Component type or format: Entity Class.

Object-Oriented Components:

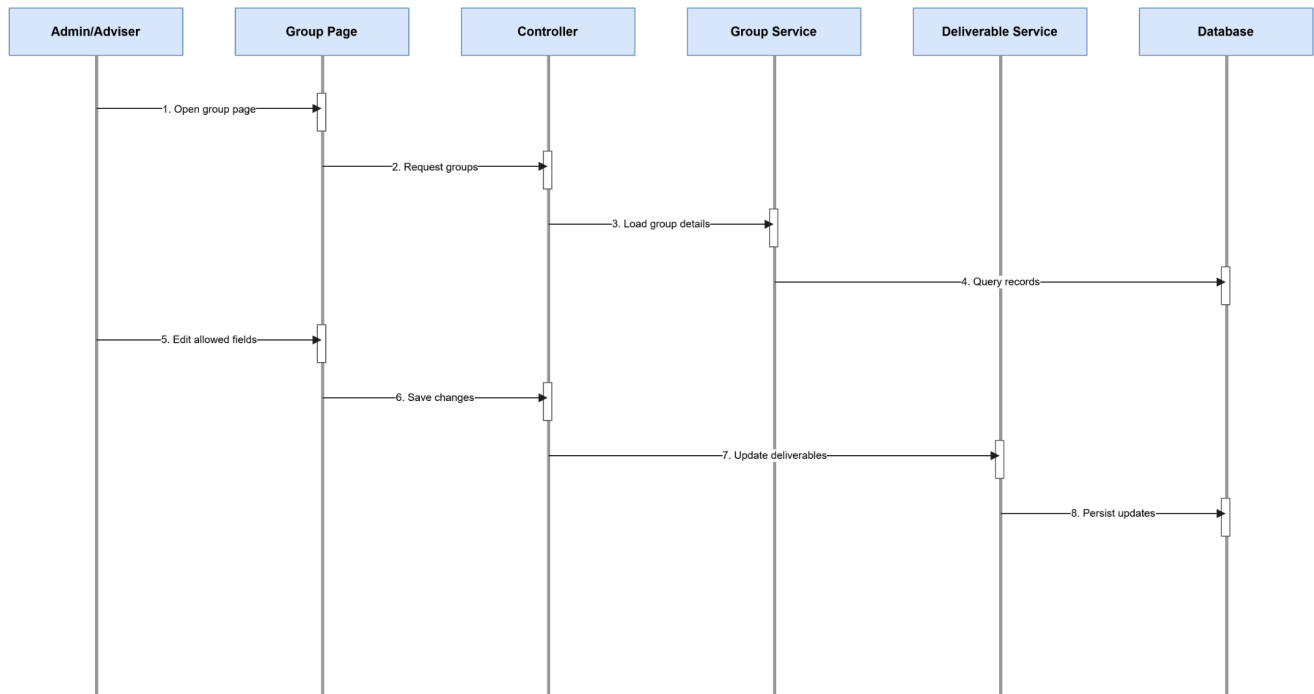
- Class Diagram:

Class - 1.2 Manage Groups and Deliverables



- **Sequence Diagram:**

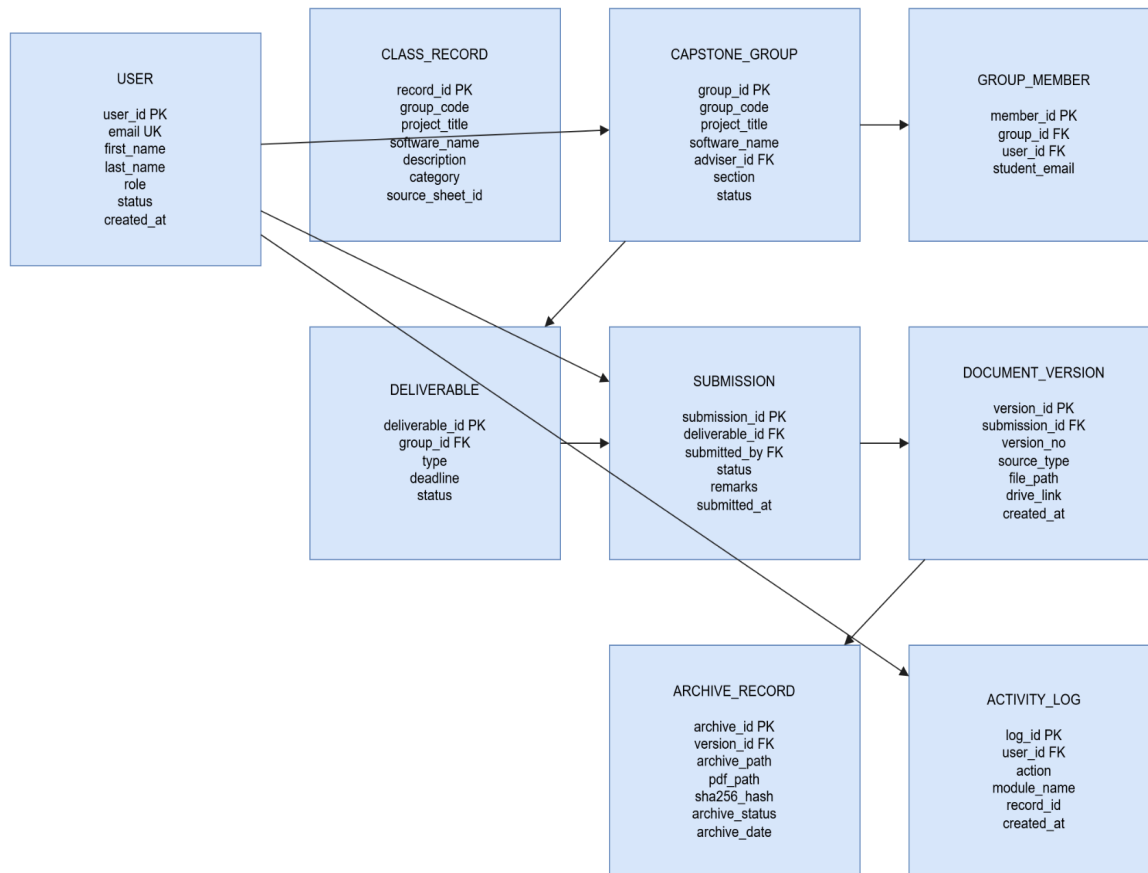
Sequence - 1.2 Manage Groups and Deliverables



Data Design:

- ERD or schema:

ERD - Global CapVault Schema



Relevant tables/entities: CAPSTONE_GROUP, GROUP_MEMBER, USER, DELIVERABLE, SUBMISSION, and ACTIVITY_LOG.

Module 2: Submission and Version Control

2.1 Submit Capstone Deliverable

User Interface Design:

Wireframe - 2.1 Submit Deliverable

CapVault	Submit Capstone Deliverable	
Sidebar Dashboard Class Records Groups Submissions Archive Reports	Deliverable Type: [SRS ▼]	Deadline: May 23, 2026 Current Status: Not Submitted
	Upload File: [Choose File]	
	Google Drive Link	
	Submission Notes	
		Submit

The Submit Capstone Deliverable interface allows Student users to select an assigned deliverable, upload a file or enter a Google Drive link, add optional notes, view deadline information, and submit the deliverable.

Front-end component(s):

- **SubmissionFormPage**
 - Description and purpose: Provides the main submission form where students select a deliverable type, upload a file, enter a Google Drive link, add notes, and submit the requirement.
 - Component type or format: React Page Component.
- **FileUploadInput**
 - Description and purpose: Allows students to attach local files for submission.
 - Component type or format: React Input Component.
- **DriveLinkInput**
 - Description and purpose: Allows students to provide a Google Drive link for document submission.
 - Component type or format: React Input Component.
- **SubmissionStatusNotice**
 - Description and purpose: Displays deadline information and indicates whether the submission will be marked as Submitted or Late.
 - Component type or format: React Display Component.

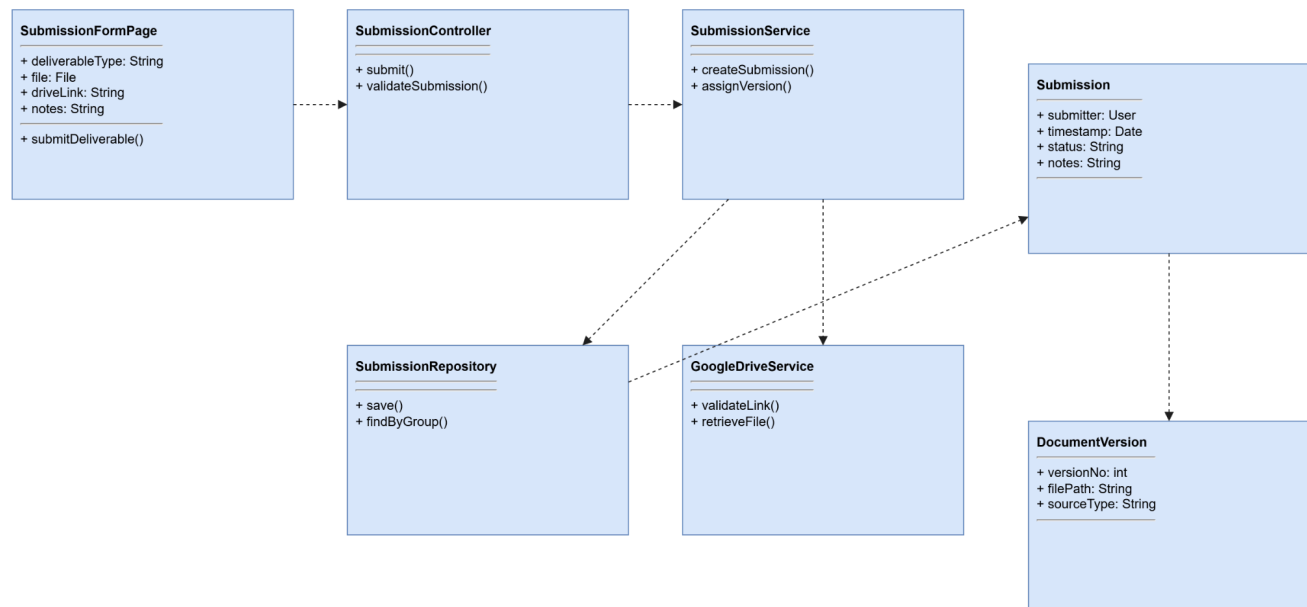
Back-end component(s):

- **SubmissionController**
 - Description and purpose: Handles API requests for creating student submissions through file upload or Google Drive link.
 - Component type or format: REST Controller.
- **SubmissionService**
 - Description and purpose: Validates submission details, creates submission records, assigns version numbers, and sets the submission status.
 - Component type or format: Service Class.
- **GoogleDriveService**
 - Description and purpose: Validates Google Drive links, retrieves accessible Drive files, and prepares Drive-linked documents for archival processing.
 - Component type or format: Service Class.
- **SubmissionRepository**
 - Description and purpose: Saves and retrieves submission records from the database.
 - Component type or format: Repository Interface.
- **Submission**
 - Description and purpose: Represents a submitted deliverable with submitter, timestamp, status, notes, and related deliverable.
 - Component type or format: Entity Class.
- **DocumentVersion**
 - Description and purpose: Represents each submitted version of a document, including version number, file path, source type, and creation date.
 - Component type or format: Entity Class.

Object-Oriented Components:

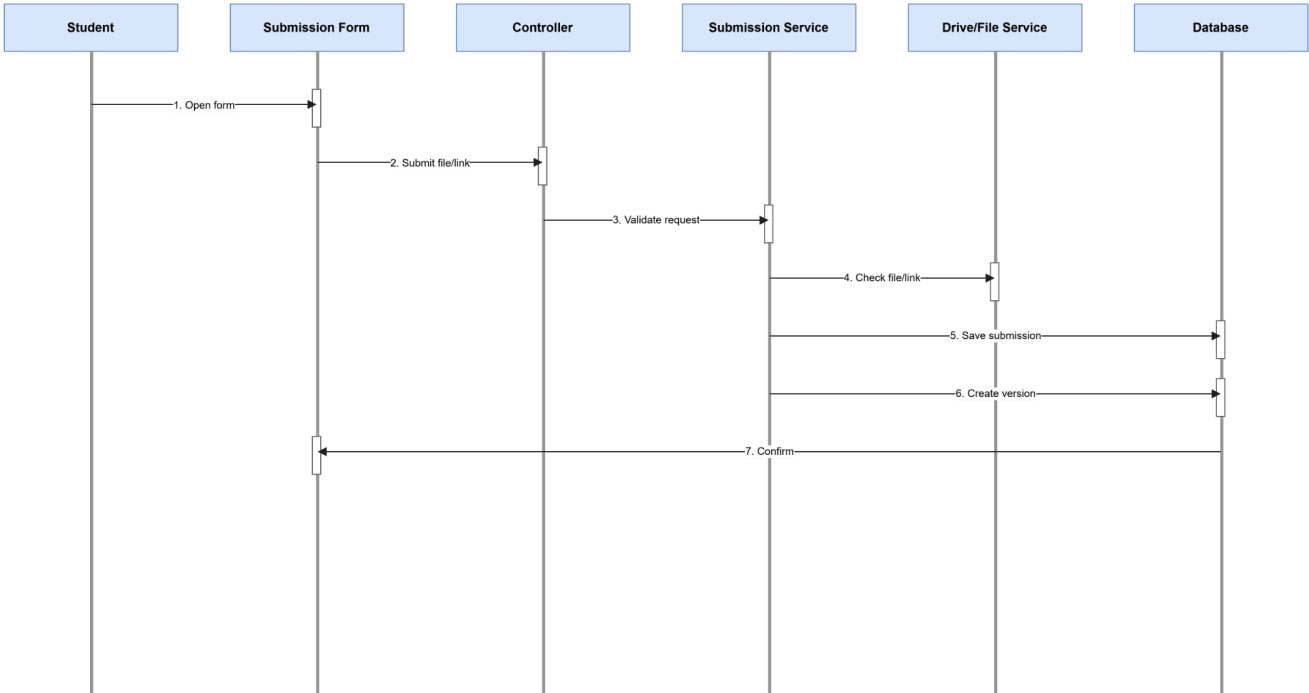
- **Class Diagram:**

Class - 2.1 Submit Capstone Deliverable



- Sequence Diagram:

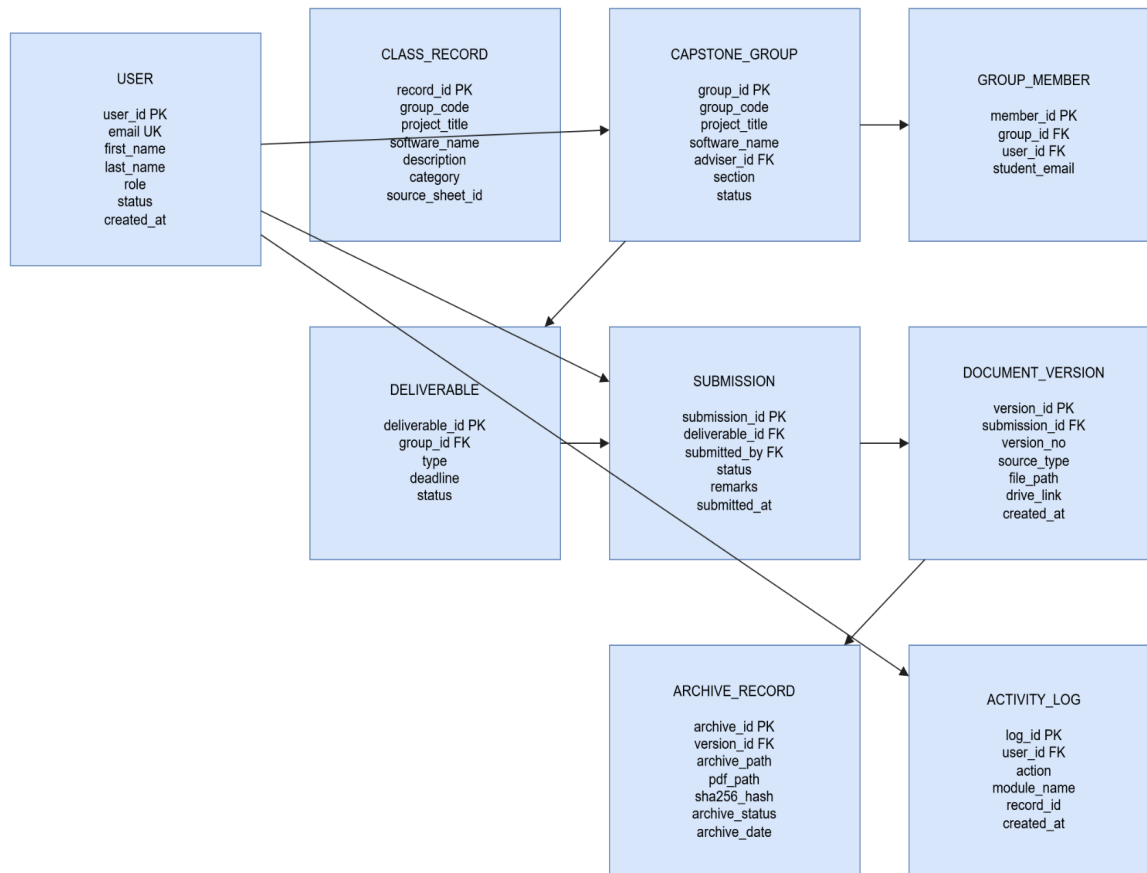
Sequence - 2.1 Submit Capstone Deliverable



Data Design:

- ERD or schema:

ERD - Global CapVault Schema



Relevant tables/entities: DELIVERABLE, SUBMISSION, DOCUMENT_VERSION, USER, and ACTIVITY_LOG.

2.1 Review Submission and Add Remarks

User Interface Design:

Wireframe - 2.2 Review Submission

CapVault

Review Submission and Add Remarks

Assigned Groups

G01 - CapVault
G02 - Project B
G03 - Project C

Submission Details

Deliverable: SRS
Version: v2
Status: Submitted
Submitted: May 22, 2026

Version History

v1 - May 20
v2 - May 22

Adviser Remarks

Status: [Needs Revision ▼]

View File

Download

Save Review

Sidebar

Dashboard
Class Records
Groups
Submissions
Archive
Reports

The Review Submission interface allows Adviser users to view assigned groups, open submitted documents, check version history, download or view files, enter remarks, and update submission status.

Front-end component(s):

- **SubmissionReviewPage**
 - Description and purpose: Provides the adviser review interface for viewing submissions, document details, version history, remarks, and status controls.
 - Component type or format: React Page Component.
- **VersionHistoryPanel**
 - Description and purpose: Displays all document versions connected to a selected submission.
 - Component type or format: React Display Component.
- **RemarksInput**
 - Description and purpose: Allows advisers to enter feedback or revision instructions for a submitted document.
 - Component type or format: React Form Component.
- **StatusDropdown**
 - Description and purpose: Allows advisers to set the submission status as Under Review, Needs Revision, Approved, Rejected, or Final.
 - Component type or format: React Select Component.

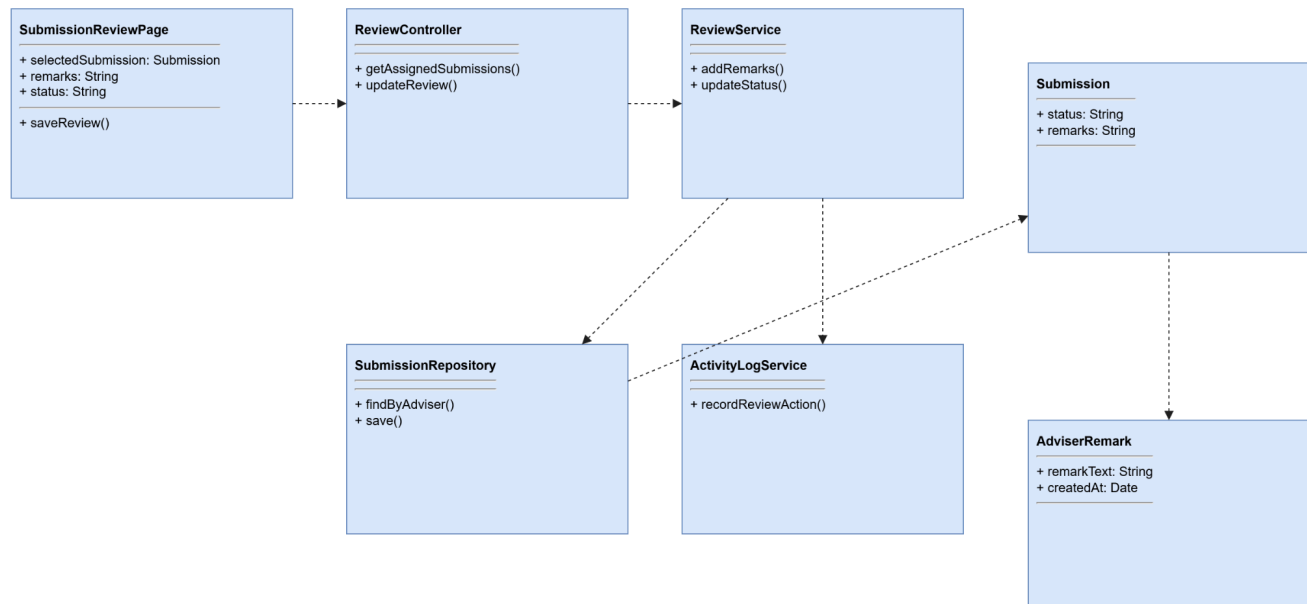
Back-end component(s):

- **ReviewController**
 - Description and purpose: Handles API requests for retrieving adviser-assigned submissions and saving remarks or status updates.
 - Component type or format: REST Controller.
- **ReviewService**
 - Description and purpose: Processes adviser review actions, updates submission status, saves remarks, and triggers activity log recording.
 - Component type or format: Service Class.
- **SubmissionRepository**
 - Description and purpose: Retrieves and updates submission records assigned to the adviser.
 - Component type or format: Repository Interface.
- **ActivityLogService**
 - Description and purpose: Records adviser review actions, status changes, and remarks updates.
 - Component type or format: Service Class.
- **Submission**
 - Description and purpose: Represents the submission being reviewed, including current status and remarks.
 - Component type or format: Entity Class.
- **AdviserRemark**
 - Description and purpose: Represents adviser remarks connected to a submission or document version.
 - Component type or format: Entity Class.

Object-Oriented Components:

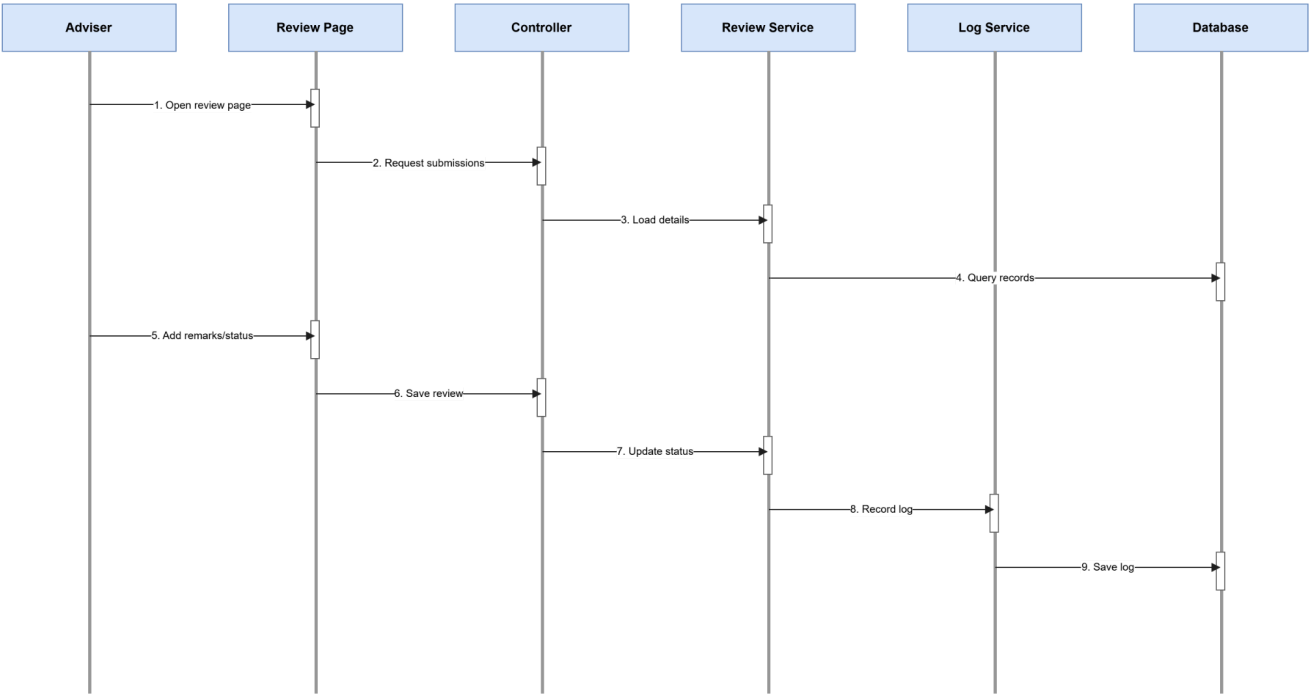
- **Class Diagram:**

Class - 2.2 Review Submission and Add Remarks



● Sequence Diagram:

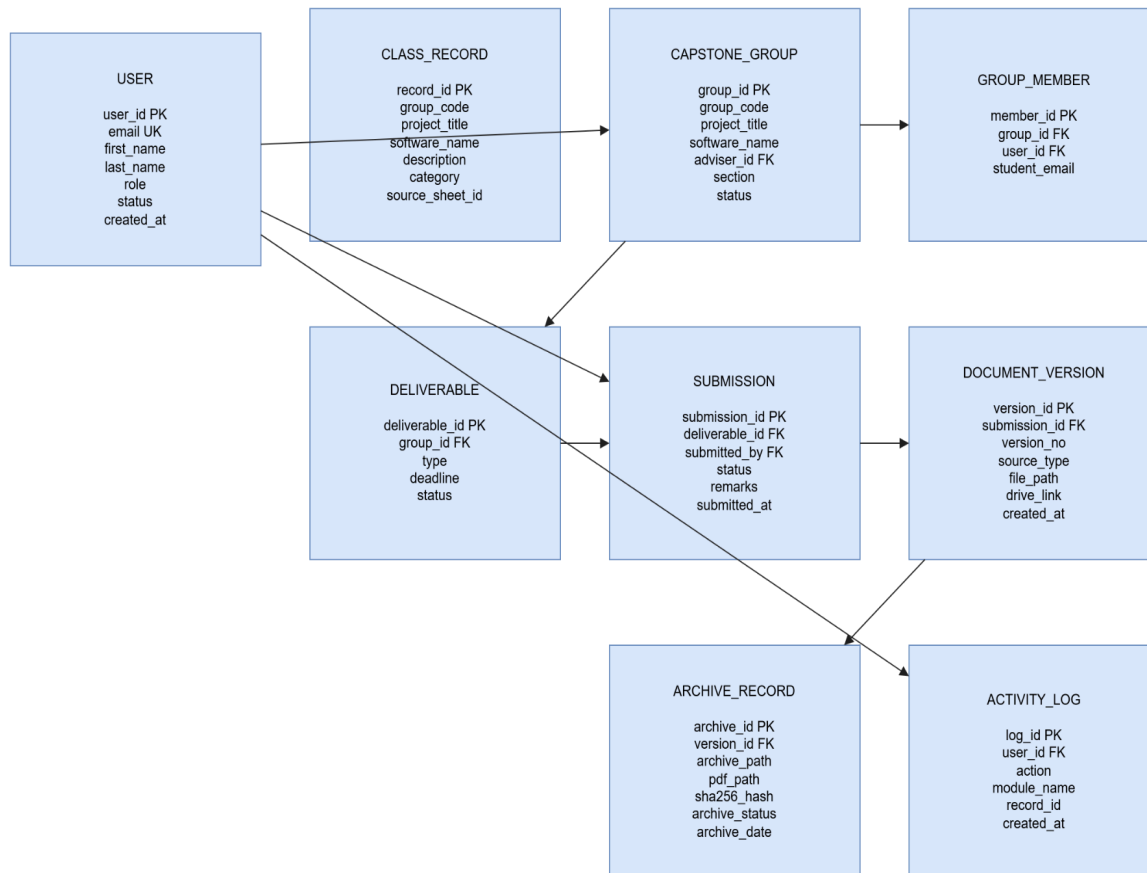
Sequence - 2.2 Review Submission and Add Remarks



Data Design:

- ERD or schema:

ERD - Global CapVault Schema



Relevant tables/entities: USER, CAPSTONE_GROUP, DELIVERABLE, SUBMISSION, DOCUMENT_VERSION, and ACTIVITY_LOG.

Module 3: Archival Vault and File Integrity Management

3.1 Archive Submitted Document

User Interface Design:

Wireframe - 3.1 Archive Document

CapVault

Archive Submitted Document

Archive Queue

SRS v2 | G01 | Ready
Proposal v3 | G02 | Ready
SDD v1 | G03 | Pending

Archive Details

Project: CapVault
Deliverable: SRS
Version: v2
Format: DOCX → PDF
Status: Archived

Metadata

Group Code: G01
Adviser: Adviser A
Date Submitted: May 22
Archive Date: May 22

View PDF

Download

Sidebar

Dashboard
Class Records
Groups
Submissions
Archive
Reports

The Archive Submitted Document interface shows archive queue records, archive details, document metadata, archive status, PDF availability, and view/download controls.

Front-end component(s):

- **ArchiveQueuePage**
 - Description and purpose: Displays pending, archived, and failed archive records. It allows authorized users to view archive details or retry failed archive processing.
 - Component type or format: React Page Component.
- **ArchiveDetailsPanel**
 - Description and purpose: Shows archive metadata such as project title, group code, adviser, deliverable type, version number, submission date, archive date, and status.
 - Component type or format: React Display Component.
- **ArchiveActionButtons**
 - Description and purpose: Provides actions such as View PDF, Download, and Retry Archive when allowed.
 - Component type or format: React Button Component.

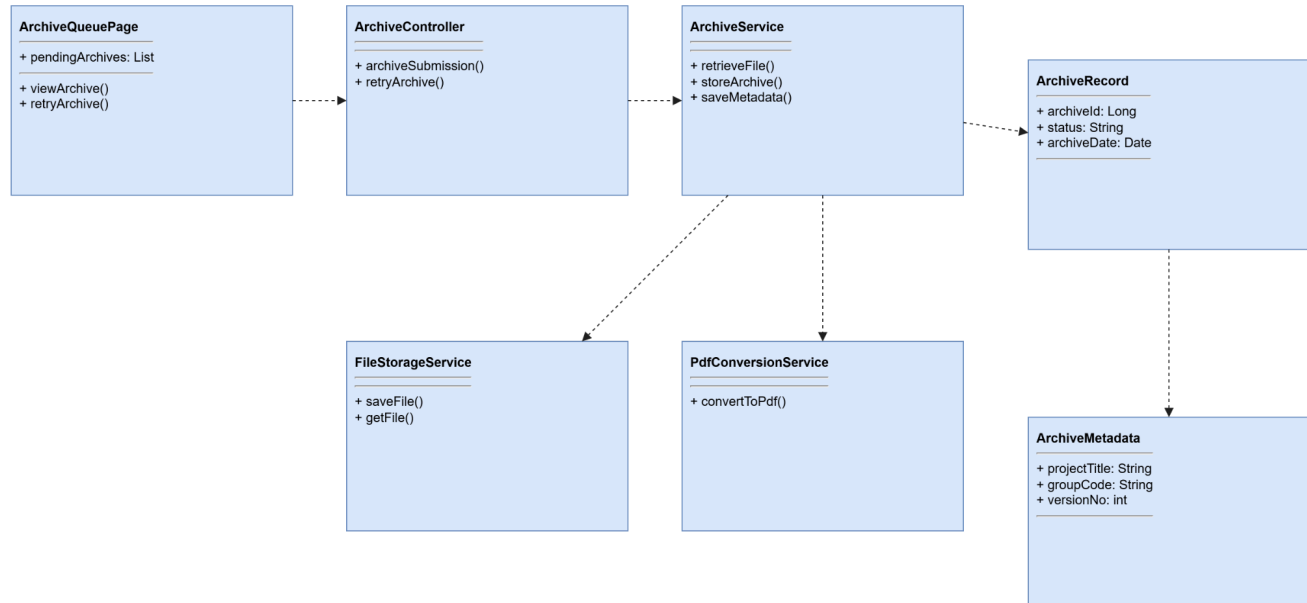
Back-end component(s):

- **ArchiveController**
 - Description and purpose: Handles API requests for archiving submissions, retrieving archive records, and retrying failed archive attempts.
 - Component type or format: REST Controller.
- **ArchiveService**
 - Description and purpose: Coordinates file retrieval, PDF conversion, archive file storage, metadata recording, and archive status updates.
 - Component type or format: Service Class.
- **PdfConversionService**
 - Description and purpose: Converts supported document formats such as DOCX or Google Docs into PDF format when applicable.
 - Component type or format: Service Class or External Conversion Utility.
- **FileStorageService**
 - Description and purpose: Stores archived files, retrieves stored files, and manages file paths for uploaded and archived documents.
 - Component type or format: Service Class.
- **ArchiveRecord**
 - Description and purpose: Represents an archived document record, including archive ID, status, archive date, file path, and hash value.
 - Component type or format: Entity Class.
- **ArchiveMetadata**
 - Description and purpose: Represents descriptive archive metadata connected to a submitted document version.
 - Component type or format: Entity Class or Embedded Entity.

Object-Oriented Components:

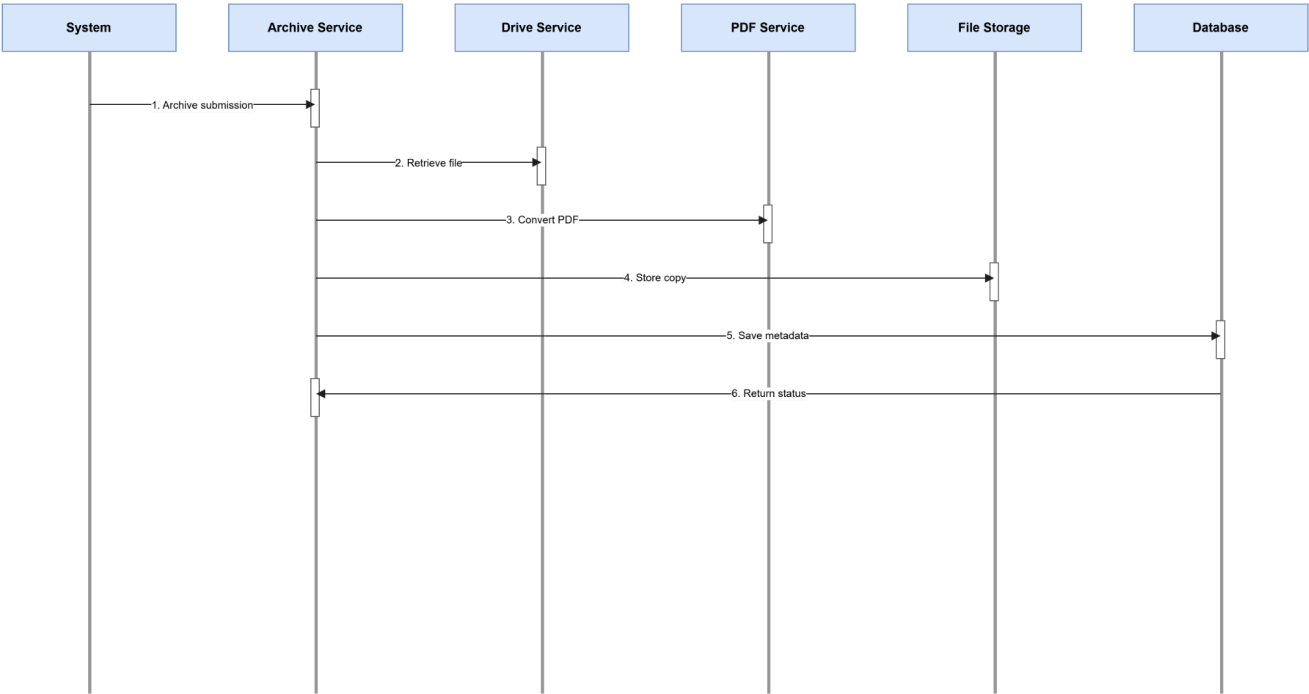
- Class Diagram:

Class - 3.1 Archive Submitted Document



● Sequence Diagram:

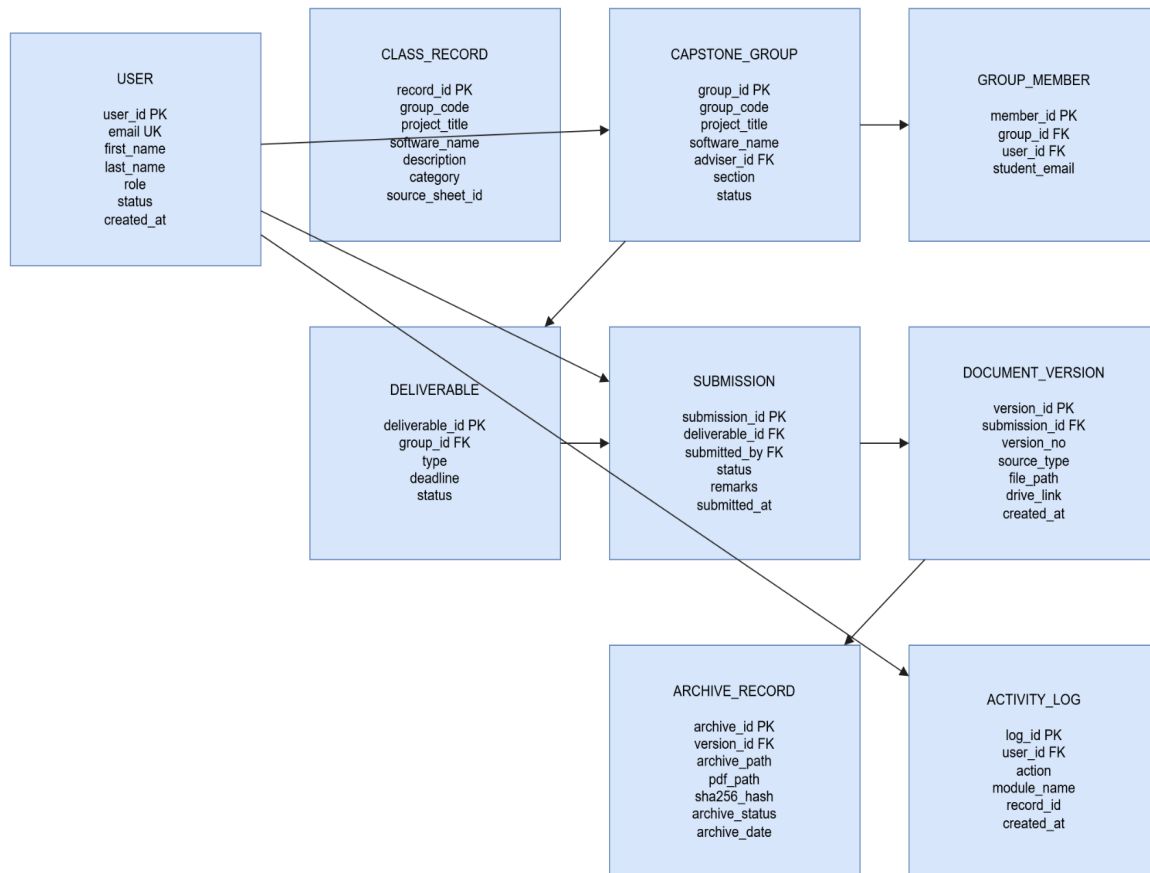
Sequence - 3.1 Archive Submitted Document



Data Design:

- ERD or schema:

ERD - Global CapVault Schema



Relevant tables/entities: SUBMISSION, DOCUMENT_VERSION, ARCHIVE_RECORD, DELIVERABLE, CAPSTONE_GROUP, and ACTIVITY_LOG.

3.2 Generate and Verify SHA-256 Hash

User Interface Design:

Wireframe - 3.2 Verify SHA-256 Hash

CapVault

Generate and Verify SHA-256 Hash

Document Info

File: SRS_v2.pdf
Group: G01
Version: v2
Archive Status: Archived

Hash Verification

Stored Hash:
8e18e413c054a9dc...

Current Hash:
8e18e413c054a9dc...

Status: Unchanged

Verify Integrity

View Log

Sidebar

Dashboard
Class Records
Groups
Submissions
Archive
Reports

The Generate and Verify SHA-256 Hash interface shows document information, stored hash, current hash, verification status, date checked, and a Verify Integrity button.

Front-end component(s):

- **IntegrityPage**
 - Description and purpose: Displays archived document integrity details and allows authorized users to request hash verification.
 - Component type or format: React Page Component.
- **HashResultPanel**
 - Description and purpose: Shows the stored hash, current hash, and verification result such as Unchanged, Modified, or Error.
 - Component type or format: React Display Component.
- **VerifyIntegrityButton**
 - Description and purpose: Triggers hash comparison for the selected archived document.
 - Component type or format: React Button Component.

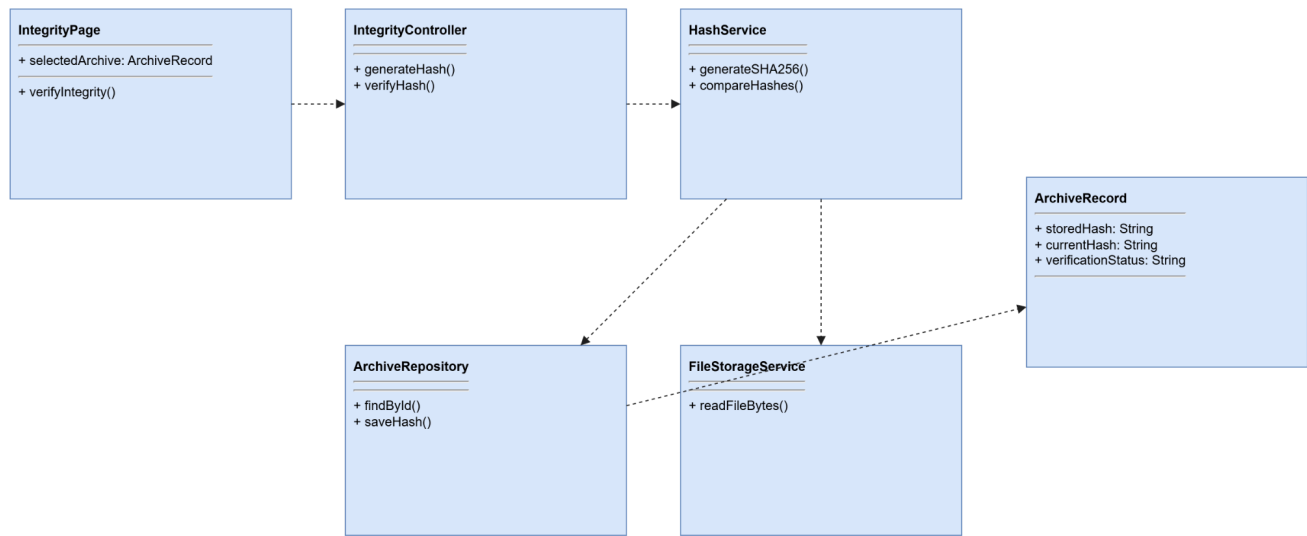
Back-end component(s):

- **IntegrityController**
 - Description and purpose: Handles API requests for generating and verifying SHA-256 hashes for archived documents.
 - Component type or format: REST Controller.
- **HashService**
 - Description and purpose: Reads file bytes, generates SHA-256 hash values, compares stored and current hash values, and returns verification results.
 - Component type or format: Service Class.
- **ArchiveRepository**
 - Description and purpose: Retrieves archive records and updates stored hash or verification status.
 - Component type or format: Repository Interface.
- **FileStorageService**
 - Description and purpose: Reads archived file bytes from storage so the hash can be generated or verified.
 - Component type or format: Service Class.
- **ArchiveRecord**
 - Description and purpose: Represents the archived document and stores its SHA-256 hash and verification status.
 - Component type or format: Entity Class.

Object-Oriented Components:

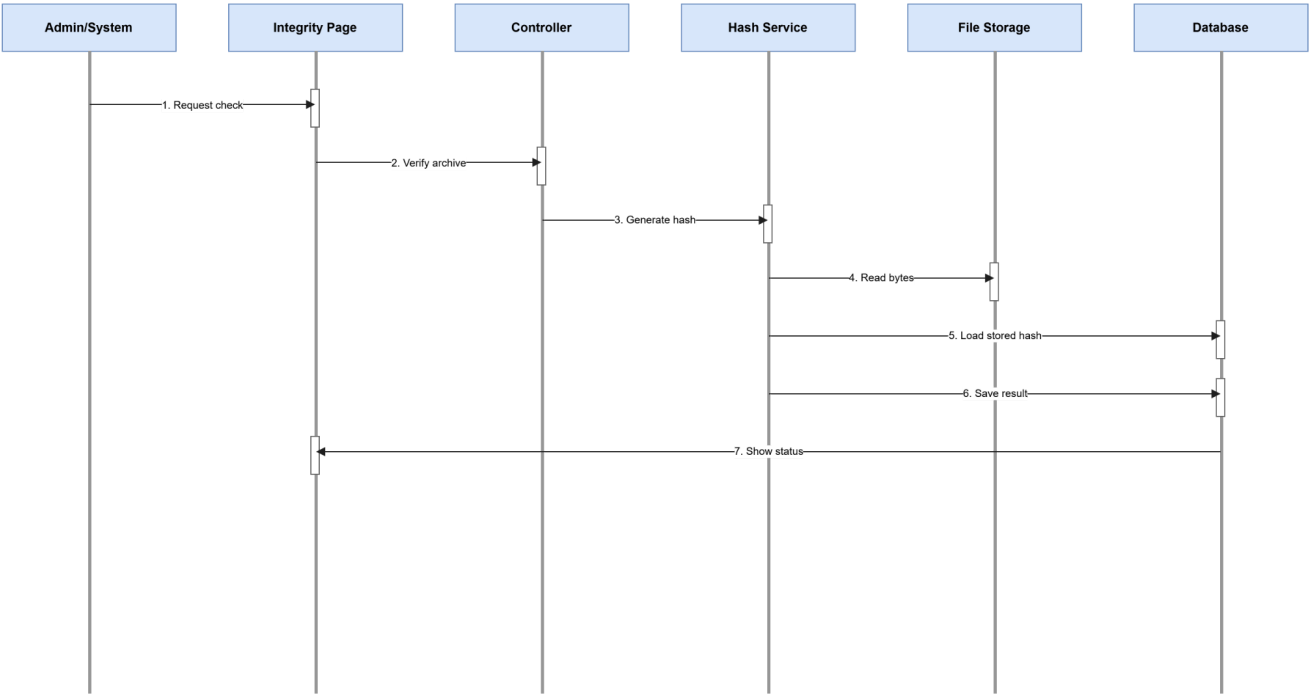
- Class Diagram:

Class - 3.2 Generate and Verify SHA-256 Hash



● Sequence Diagram:

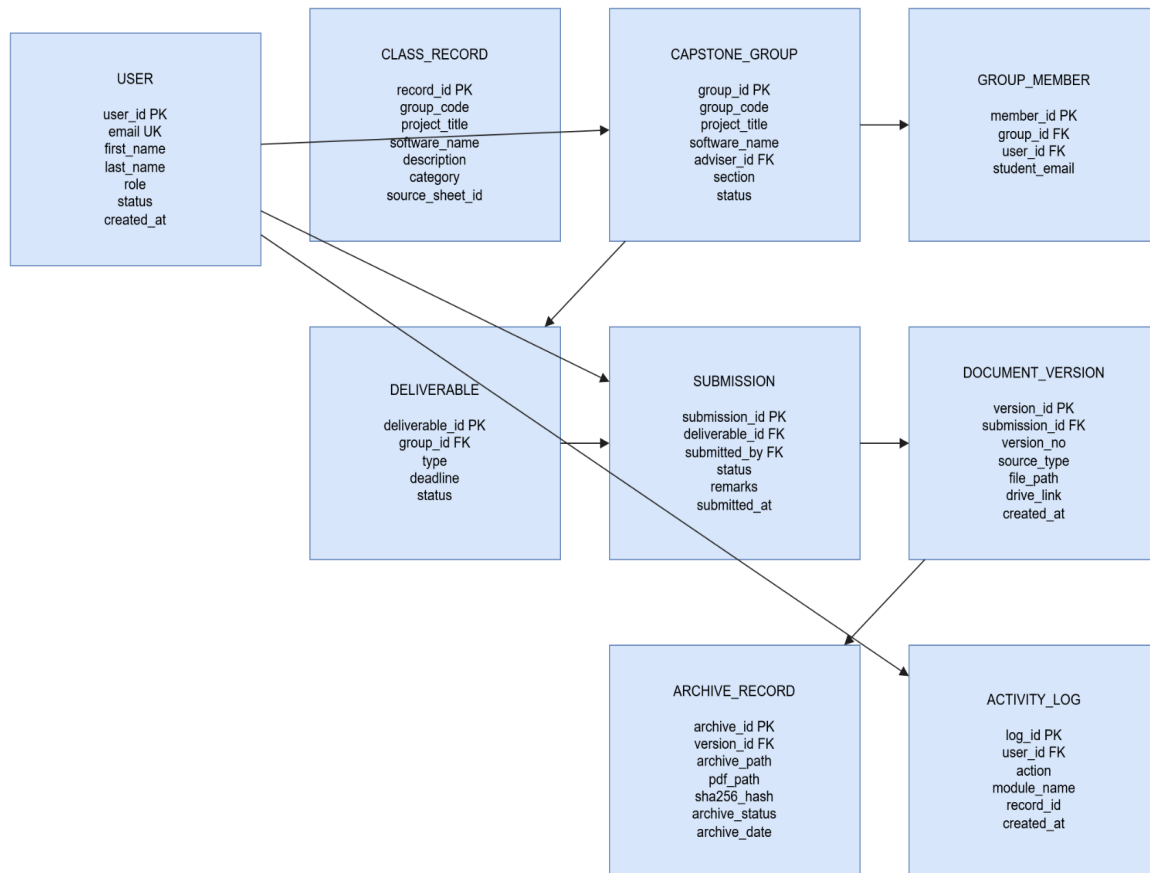
Sequence - 3.2 Generate and Verify SHA-256 Hash



Data Design:

- ERD or schema:

ERD - Global CapVault Schema



Relevant tables/entities: ARCHIVE_RECORD, DOCUMENT_VERSION, SUBMISSION, and ACTIVITY_LOG.

Module 4: Search, Retrieval, and Reporting

4.1 Search and Retrieve Archived Records

User Interface Design:

Wireframe - 4.1 Search Records

CapVault

Search and Retrieve Archived Records

Search project / group / student

Deliverable Type ▼

Status ▼

Results

G01 | CapVault | SRS | v2 | Final | May 22
G01 | CapVault | Proposal | v3 | Approved | May 20
G02 | Project B | SDD | v1 | Under Review | May 21

Selected Record Metadata

Project Title: CapVault
Group Code: G01
Adviser: Adviser A
Version: v2
SHA-256: 8e18...

View

Download

Sidebar

Dashboard
Class Records
Groups
Submissions
Archive
Reports

The Search and Retrieve Archived Records interface allows authorized users to search and filter archived records by project title, group code, student name, adviser, section, deliverable type, status, version, and date.

Front-end component(s):

- **ArchiveSearchPage**
 - Description and purpose: Provides search and filtering controls for archived capstone documents.
 - Component type or format: React Page Component.
- **ArchiveResultsTable**
 - Description and purpose: Displays matching archive records, including project title, group code, deliverable type, version number, status, and archive date.
 - Component type or format: React Table Component.
- **ArchiveMetadataPanel**
 - Description and purpose: Shows detailed metadata and version information for the selected archive record.
 - Component type or format: React Display Component.
- **DownloadButton**
 - Description and purpose: Allows authorized users to download accessible archived files.
 - Component type or format: React Button Component.

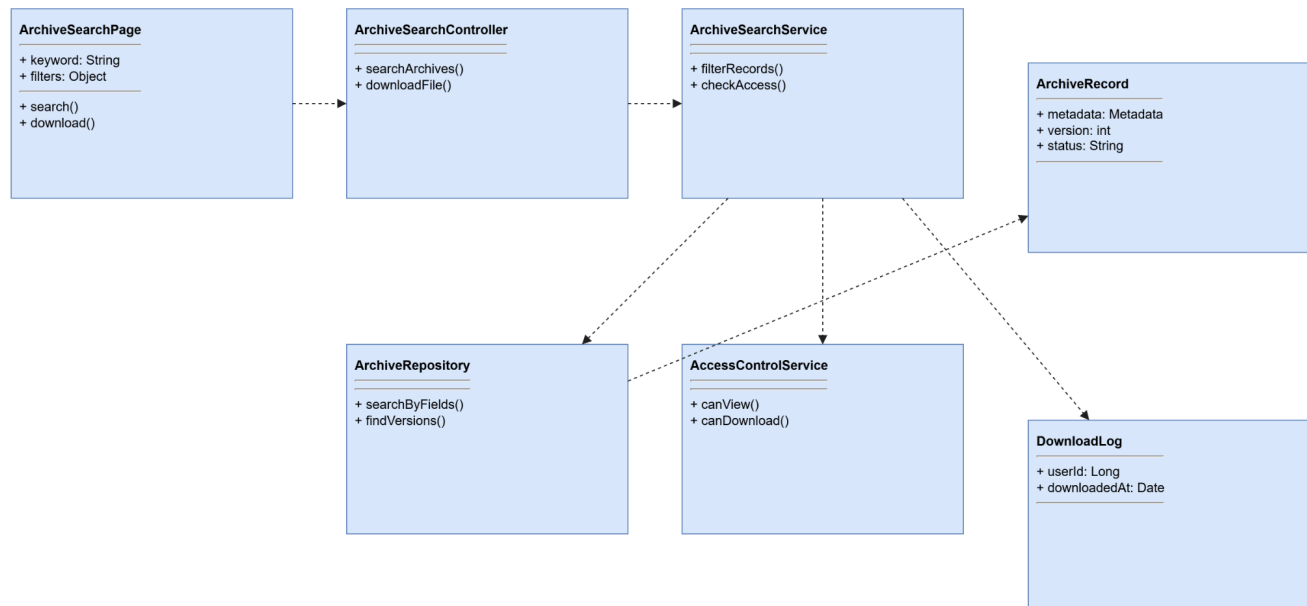
Back-end component(s):

- **ArchiveSearchController**
 - Description and purpose: Handles API requests for searching, filtering, viewing, and downloading archived records.
 - Component type or format: REST Controller.
- **ArchiveSearchService**
 - Description and purpose: Applies filters, retrieves matching archive records, and checks user access permissions.
 - Component type or format: Service Class.
- **ArchiveRepository**
 - Description and purpose: Searches archive records by metadata fields and retrieves related document versions.
 - Component type or format: Repository Interface.
- **AccessControlService**
 - Description and purpose: Checks whether the logged-in user can view or download the requested archive record.
 - Component type or format: Service Class.
- **ArchiveRecord**
 - Description and purpose: Represents archived document data and metadata used in search results.
 - Component type or format: Entity Class.
- **DownloadLog**
 - Description and purpose: Records download activity for accountability and audit purposes.
 - Component type or format: Entity Class.

Object-Oriented Components:

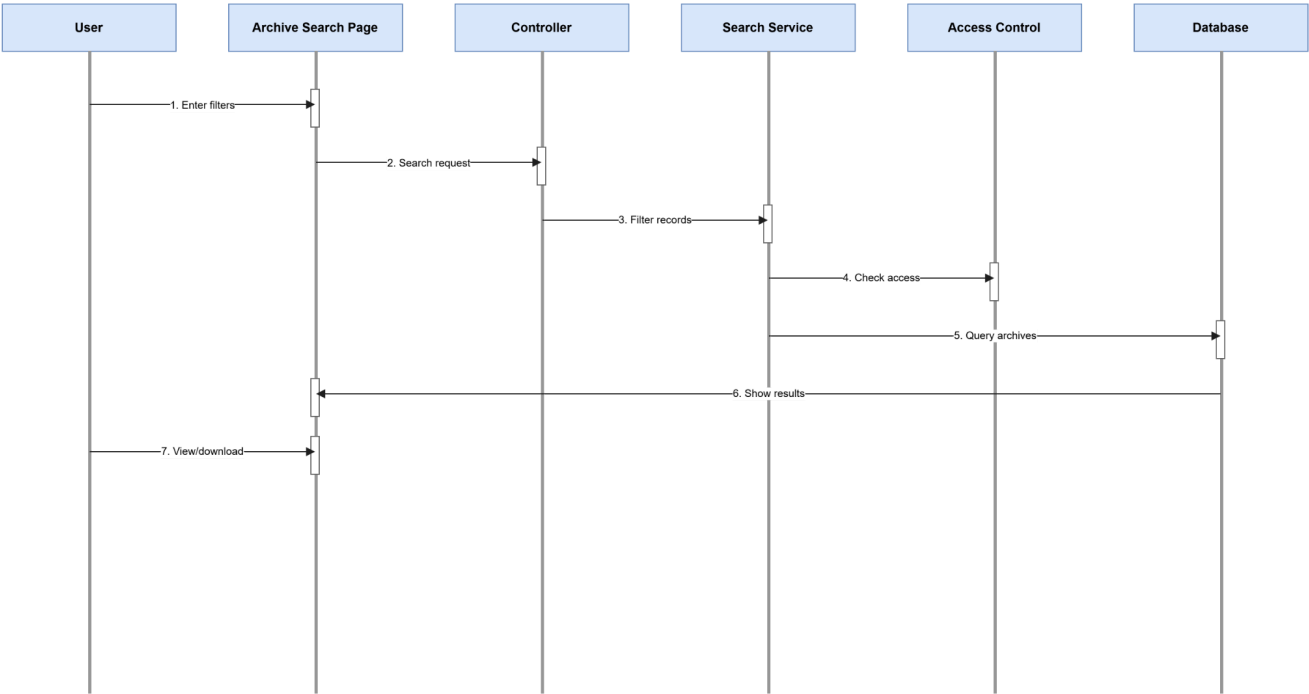
- Class Diagram:

Class - 4.1 Search and Retrieve Archived Records



● Sequence Diagram:

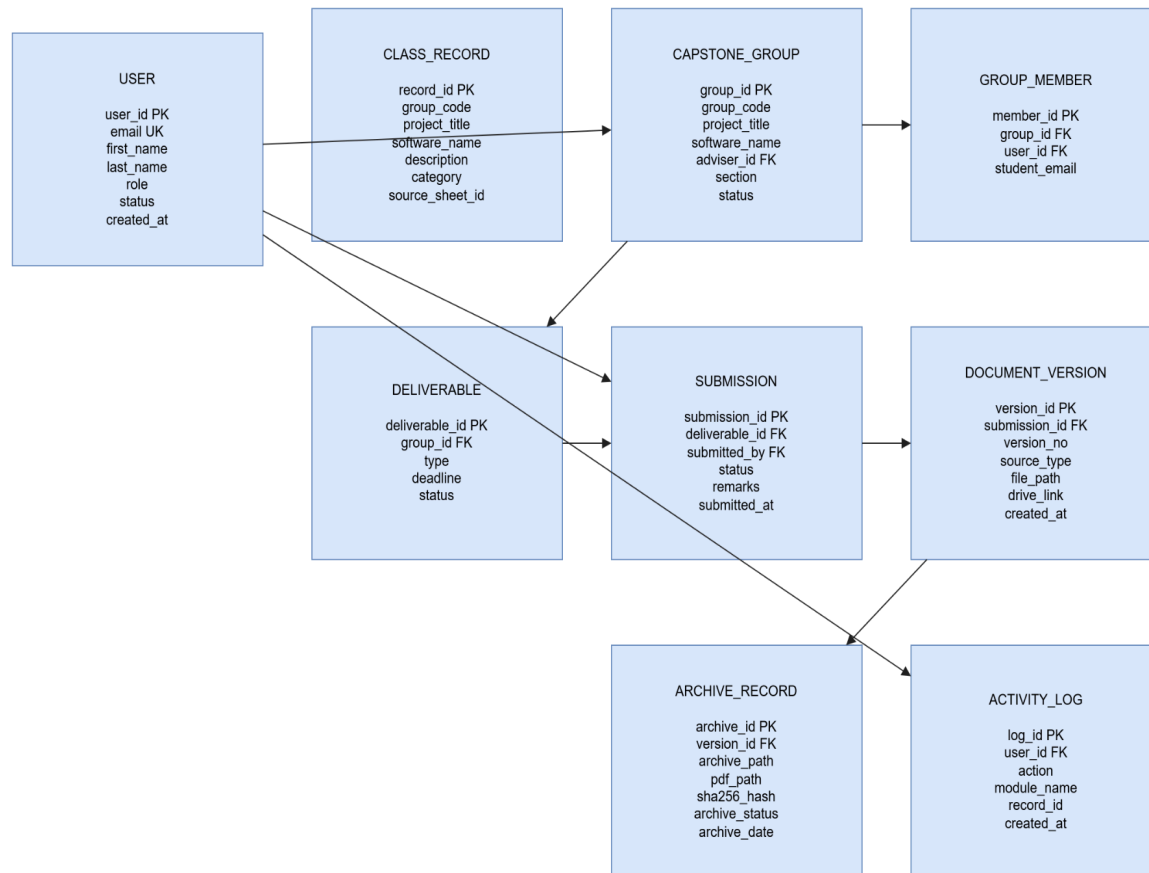
Sequence - 4.1 Search and Retrieve Archived Records



Data Design:

- ERD or schema:

ERD - Global CapVault Schema

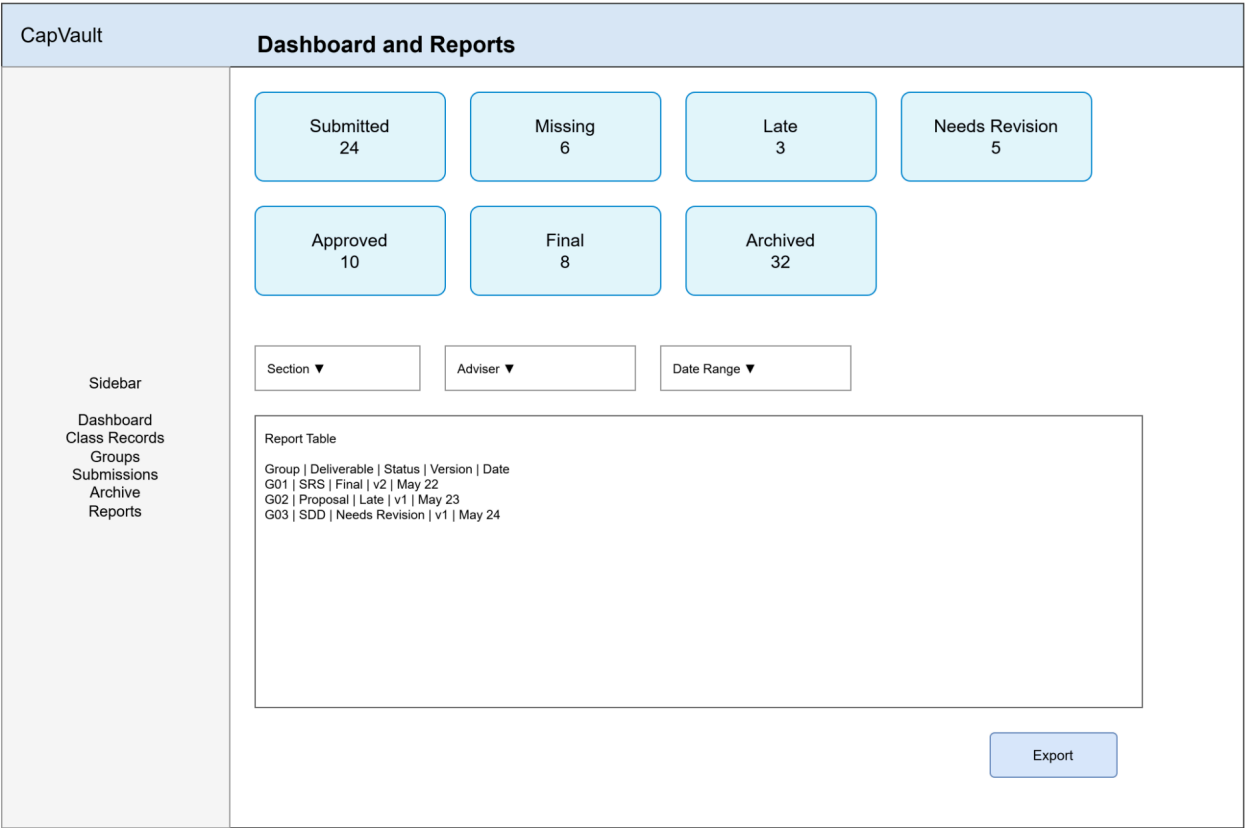


Relevant tables/entities: ARCHIVE_RECORD, DOCUMENT_VERSION, SUBMISSION, CAPSTONE_GROUP, USER, and ACTIVITY_LOG.

4.2 Generate Dashboard and Reports

User Interface Design:

Wireframe - 4.2 Dashboard and Reports



The Generate Dashboard and Reports interface displays dashboard cards, report filters, and report tables for submitted, missing, late, needs revision, approved, final, and archived deliverables.

Front-end component(s):

- **DashboardReportsPage**
 - Description and purpose: Provides dashboard summaries and report controls for Admin and Adviser users.
 - Component type or format: React Page Component.
- **DashboardSummaryCards**
 - Description and purpose: Displays counts for submitted, missing, late, needs revision, approved, final, and archived documents.
 - Component type or format: React Card Component.
- **ReportFilterPanel**
 - Description and purpose: Allows users to filter reports by section, adviser, group, deliverable type, status, or date.
 - Component type or format: React Form Component.
- **ReportTable**
 - Description and purpose: Displays report rows based on selected filters.
 - Component type or format: React Table Component.

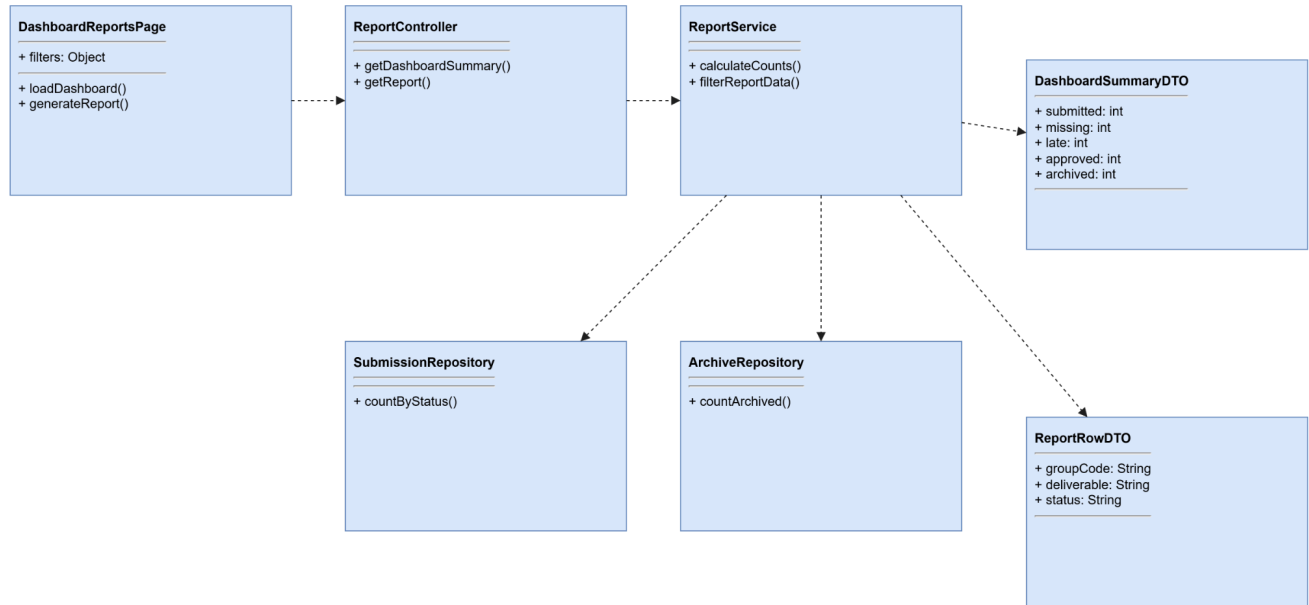
Back-end component(s):

- **ReportController**
 - Description and purpose: Handles API requests for dashboard summaries and filtered report data.
 - Component type or format: REST Controller.
- **ReportService**
 - Description and purpose: Calculates counts, applies filters, and prepares report data for dashboard and reporting pages.
 - Component type or format: Service Class.
- **SubmissionRepository**
 - Description and purpose: Counts and retrieves submission records by status, group, deliverable, adviser, section, or date.
 - Component type or format: Repository Interface.
- **ArchiveRepository**
 - Description and purpose: Counts archived records and retrieves archive-related data for reports.
 - Component type or format: Repository Interface.
- **DashboardSummaryDTO**
 - Description and purpose: Transfers dashboard count data from the backend to the frontend.
 - Component type or format: Data Transfer Object.
- **ReportRowDTO**
 - Description and purpose: Transfers filtered report row data from the backend to the frontend.
 - Component type or format: Data Transfer Object.

Object-Oriented Components:

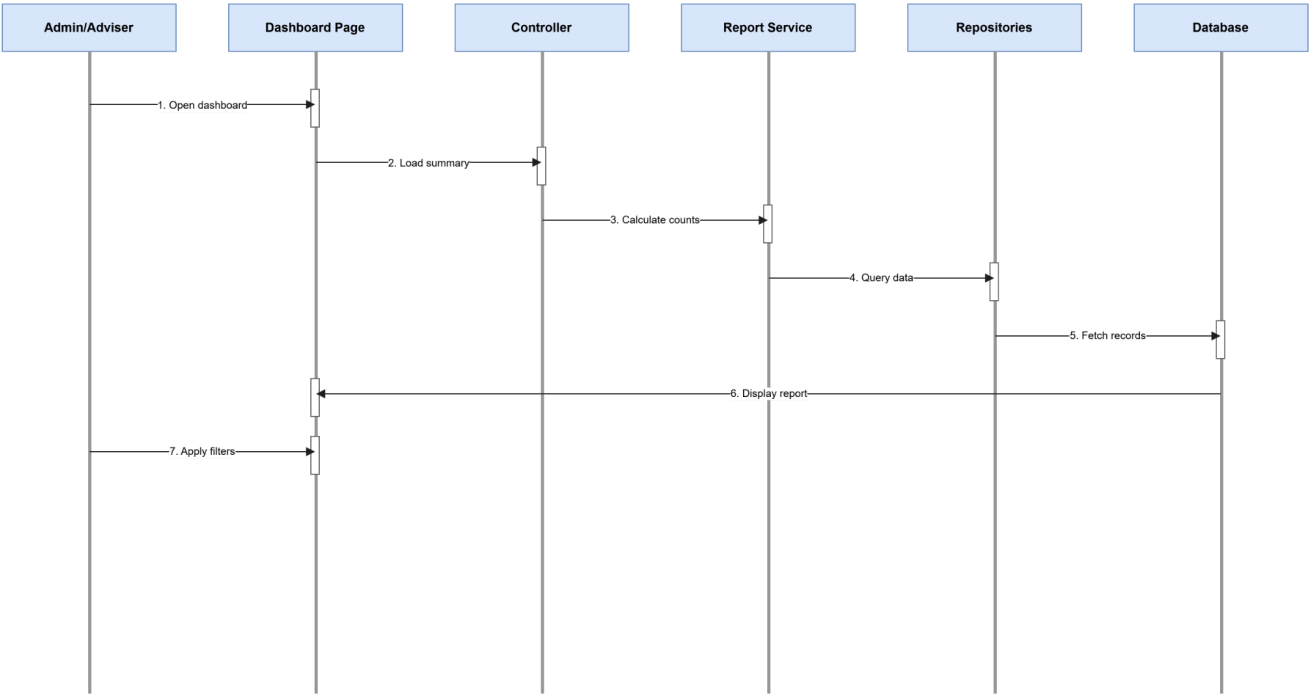
- Class Diagram:

Class - 4.2 Generate Dashboard and Reports



● Sequence Diagram:

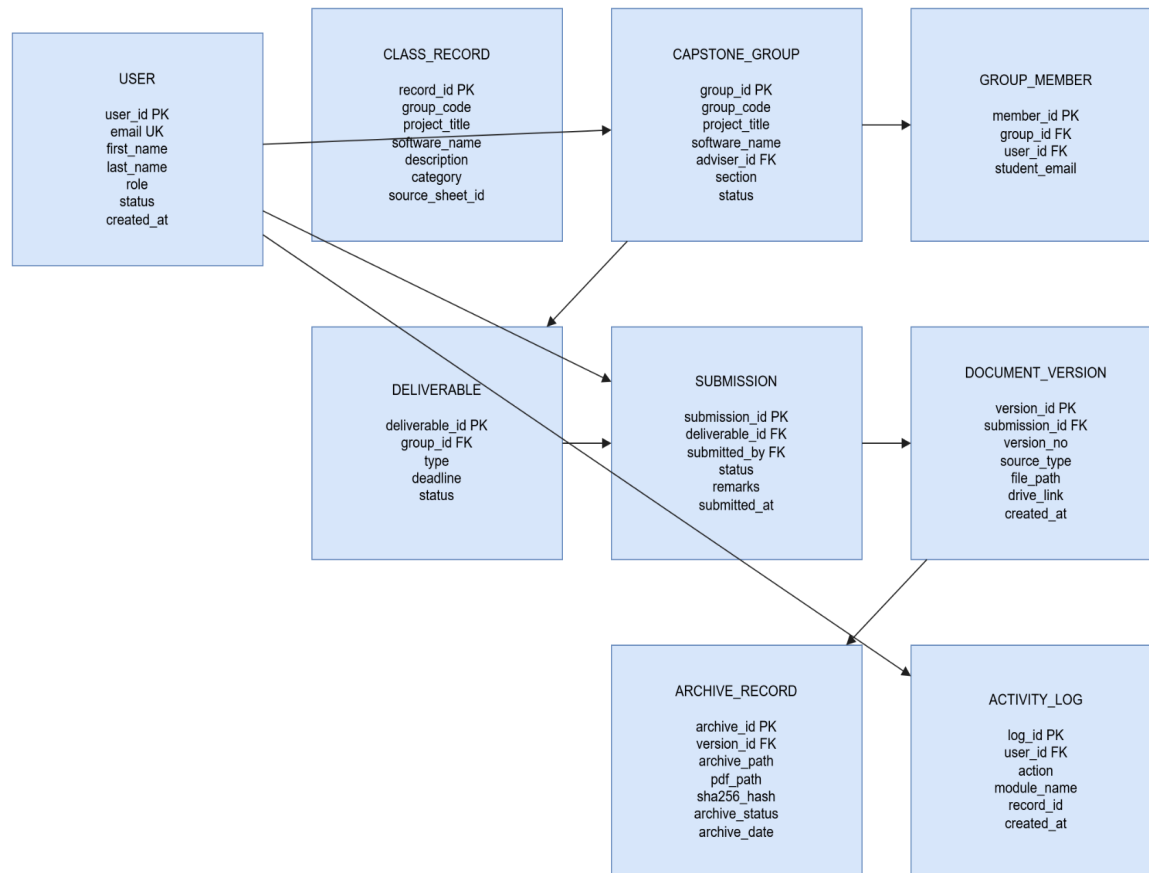
Sequence - 4.2 Generate Dashboard and Reports



Data Design:

- ERD or schema:

ERD - Global CapVault Schema



Relevant tables/entities: CAPSTONE_GROUP, DELIVERABLE, SUBMISSION, DOCUMENT_VERSION, ARCHIVE_RECORD, USER, and ACTIVITY_LOG.